Predefined Avatars System — Research & Implementation Guide

AI Video Generation Platform (India-Centric)

End-to-End Character Identity Pipeline for Wan 2.2 Consistency

Document Date: 09 October 2026 | Aligned with Detailed Project Report (September 2026) | Modules 1–5, 7

1. Overview & Alignment with DPR

The Detailed Project Report (DPR) defines the high-level flow: User selects or generates a character → unique **Character ID** is created → references are stored → Character ID + prompt flow through JEV → Gemini scene orchestration → VideoBlueprint → Wan 2.2 (especially I2V-A14B and Animate-14B) → Character ID tracking + consistency checks (Module 5 QA with AdaFace / DINOv3) → selective regeneration → FFmpeg assembly.

This document provides a complete, production-ready design for **10 predefined avatars**: how to create them, integrate them into the application, assign and track Character IDs, build the identity pipeline, pass avatars with prompts, and maintain identity stability across short (~5-second) Wan 2.2 clips. The design draws on research of Higgsfield Soul ID, AI Films Studio, FilmSpark, Runway, Kling, Seedance, Veo, InstantID, IP-Adapter, and Wan-Animate.

2. Creating the 10 Predefined Avatars

Best practice from industry (Higgsfield Soul ID, AI Films Studio, FilmSpark, Runway, Kling, Seedance, InstantID / IP-Adapter patterns, Wan-Animate):

2.1 Character Sheet Requirements

- Create **canonical multi-angle character sheets** — the single most important asset.
- Generate or curate 8–20 high-quality images per avatar covering:
- Neutral front portrait (even lighting, high resolution)
- Three-quarter left and right views
- Profile left and right
- Full body front / back
- Expression variants (neutral, smile, speaking mouth shapes)
- Optional: different lighting or simple wardrobe variants (keep base clothing fixed for the predefined set)
- Prefer photorealistic or stylized **Indian-centric faces** (aligns with DPR India-centric requirement).

2.2 Generation Tools

- **GPT-Image-2** (primary per Module 3) or FLUX.2 [pro] / Nano Banana 2 for fast iteration.
- InstantID, IP-Adapter, PhotoMaker, or ConsistentID (open-source) for identity lock during sheet creation.
- Character-sheet prompt pattern: “character reference sheet of [detailed description], pure clean deep neutral grey background, editorial layout, identical identity across panels…”

2.3 What to Store per Avatar

- Primary reference image (highest-quality frontal)
- Full multi-angle sheet
- Face embedding (AdaFace or ArcFace — matches Module 5)
- Optional lightweight LoRA / identity adapter if later fine-tuned
- Metadata: age range, gender, ethnicity/region, skin tone, hair, distinguishing features, clothing default, style tags

2.4 Storage Recommendation (DPR Modules 7 & 8)

• **PostgreSQL**: Character table (character_id, name, type="predefined", reference_asset_ids, appearance_metadata JSON, version, consent_status=system, embedding_vector or pointer).
• **Private S3-compatible object storage**: actual images and any derived assets.
Name them clearly (e.g., Avatar_01_Priya, Avatar_02_Arjun, …) and expose them in the frontend avatar selection UI (owned by K. Kusuma per the team table).

3. Assigning and Tracking Character ID

Follow the exact DPR pattern:

```
Character ID = UUID (or short prefixed UUID like char_01_a1b2c3d4)
```

• On avatar selection or generation → create Character record → assign unique Character ID → store reference assets linked to that ID.
• Persist in every related table: Project, PromptRevision, Blueprint, Scene/Shot, Asset, GenerationJob, QAReport.
• Every generated keyframe, clip, and final render carries the Character ID in metadata.
• Backend (FastAPI + PostgreSQL) enforces ownership and retrieves the correct reference assets by Character ID whenever a job is dispatched.

This is identical to how AI Films Studio uses `@character:{id}`, Higgsfield uses `soul_id` / Element tags, and FilmSpark uses Actor Profiles.

4. Identity Pipeline (End-to-End)

Recommended layered pipeline combining DPR Modules 3/4/5 with industry best practices:

1. Selection / Creation

User picks one of the 10 predefined avatars (or generates new). System loads Character ID + primary reference + multi-angle sheet + face embedding.

2. Reference Conditioning (Module 3)

Generate keyframes conditioned on the Character ID references using GPT-Image-2 + IP-Adapter / InstantID / ControlNet. QA gate (Module 5 Checkpoint 1) with AdaFace + DINOv3 before any video generation.

3. Prompt Augmentation

Inject a locked identity block into every prompt and into the VideoSpec / VideoBlueprint.

4. Video Generation with Wan 2.2 (Module 4)

Prefer **I2V-A14B** (image + prompt) or **Animate-14B** (character image + motion video) for strongest identity lock. Always feed the same canonical reference image(s) of the Character ID. For multi-clip sequences, use the last good frame of the previous clip as additional reference when possible. T2V-A14B only when no strong visual reference is available (weaker consistency).

5. Consistency Tracking & QA (Module 5)

Per-clip: AdaFace identity score vs original Character embedding, DINOv3 appearance, RAFT temporal, CLIP semantic, LPIPS/SSIM. Decision: PASS / AUTO_RETRY (regenerate only the failed segment with same Character ID references) / HUMAN_REVIEW. Cross-clip continuity checks using the Character ID as the join key.

6. Rendering & Delivery

Only approved clips carrying the same Character ID are composed. Final manifest records Character ID(s) used.

This matches Higgsfield's Soul ID (train once → reuse identity), Wan-Animate's design (reference image = identity anchor, driving video = motion), InstantID / IP-Adapter (zero-shot face embedding injection), and the multi-reference + keyframe-first workflows used by Seedance, Kling, Runway, and Veo.

5. Passing the Avatar with the Prompt & Consistency Across ~5 s Clips

Proven prompt construction pattern used across platforms:

```
[IDENTITY BLOCK - FIXED]
Character ID: char_01_priya
A young Indian woman, [exact physical description from Character metadata], consistent facial
features, same bone structure, same skin tone, same hair...

[REFERENCE]
Use the attached reference image(s) of Character ID char_01_priya as the primary identity
anchor.

[VARIABLE]
[action / expression / camera / environment / style]
```

• In the API / VideoSpec: include `character_ids: ["char_01_priya"]` and `reference_asset_uris`.
• JEV + Gemini 3.8 Flash (Modules 1/2) always expand the prompt while preserving the identity block and Character ID.
• For every 5-second Wan 2.2 clip: always pass the same Character ID reference image(s); prefer I2V or Animate mode; keep seed / generation settings reproducible where possible; run Module 5 QA against the original Character embedding; if identity score drops below threshold → AUTO_RETRY with the same references.
• For longer videos: generate independent 4–8 s clips, then chain last-frame → next first-frame when the model supports it, or simply compose in FFmpeg after QA passes.

Why this works well with Wan 2.2

• Wan 2.2 Animate-14B and I2V-A14B are explicitly designed for character animation / replacement with a reference image as the identity anchor.
• VACE can be used for further control / editing while preserving identity.
• Short clips (5 s target) are ideal — longer generations increase drift risk.

6. How Other Platforms Solve Character Consistency

Comparative research summary of leading platforms and open-source approaches:

Platform / Approach	Core Mechanism	Character ID / Persistence	Multi-Clip Strength
Higgsfield Soul ID	Train identity on 20–80 photos → persistent Soul ID	Soul ID / Element tag	Excellent — locked across Kling, Veo, Seedance, WAN
AI Films Studio	Character sheet + unique ID	@character:{id} injected into prompt	Strong with Seedance
FilmSpark / Sozee / Astorie	Actor Profile with multi-reference images	Persistent profile + reference set	Multi-angle + multi-shot
Runway / Kling / Veo / Seedance	Reference image(s) per generation (1–9)	No permanent ID; reference conditioning	Good for short sequences; drift over many clips
HeyGen / Synthesia	Pre-trained digital twin / stock avatar	Avatar ID + voice clone	Excellent for talking-head; less cinematic flexibility
Open-source (InstantID, IP-Adapter, ConsistentID, ID-Animator, Wan-Animate)	Face embedding + ControlNet / adapter injection	Manual reference management	Fully controllable; matches self-hosted direction
Wan 2.2 specifically	Reference image (identity) + driving video (motion) or I2V	External Character ID you manage	Best when always supplying the same reference image

Table 1. Platform comparison of character consistency mechanisms (research as of 2026).

7. Recommended Implementation Steps (Team Mapping)

Owner / Module	Responsibility
K. Kusuma + Module 3 (K. Shiva)	Design and generate the 10 character sheets + embeddings. Store in Character table + S3.

Owner / Module	Responsibility
Frontend (N. Vijay)	Avatar picker UI that returns Character ID on selection.
Module 1/2 (Kanaka Raju + Vaishnavi)	Always inject Character ID + identity block into VideoSpec and VideoBlueprint.
Module 4 (A. Srimanth)	Wan adapter must accept Character ID → resolve to reference images → prefer I2V / Animate modes.
Module 5 (M. Kannan)	Use Character ID to fetch the original embedding for every QA check. Log identity scores per clip.
Backend (Sai Harshitha)	Character entity already sketched in the DPR data model — implement fully with versioning and audit.

Table 2. Team ownership for predefined avatar system implementation.

8. Summary & Key Takeaways

This approach delivers the same consistency power as Higgsfield Soul ID and commercial avatar platforms, while remaining fully aligned with the modular, provider-agnostic, QA-gated architecture and Wan 2.2 stack defined in the DPR. Key principles:

- **Create once, reuse forever** — high-quality multi-angle character sheets + face embeddings stored under a stable Character ID.
- **Always condition generation** — pass the same reference image(s) into every Wan 2.2 I2V or Animate call.
- **Lock identity in the prompt** — fixed identity block + Character ID in VideoSpec / VideoBlueprint.
- **Gate with QA** — AdaFace / DINOv3 identity checks with AUTO_RETRY on the same Character ID references.
- **Prefer short clips** — 4–8 second generations minimize temporal drift; compose longer videos after QA.
- **Track everywhere** — Character ID lives in every database table and every generation/render metadata record.

This research addendum is intended to guide Phase 1–2 implementation of the predefined avatar subsystem. Exact model checkpoints, embedding thresholds, and commercial licences must be revalidated at production time.